HACKATHON BUILD / PERSON 2 / BACKEND

Ledger of the Undead

Backend technical dossier

Fraud-ring detection, transparent risk scoring
and guardrailed AI explanations - explained line by line.

```
GET /alerts  ->  { alerts[], count, source, cached }
risk = normalized feature x weight, five features, 0-100
Gemini -> number guardrail -> template (always available)
```

Companion to README.md, RUN.md and app/ - 24 October 2026

FastAPI + Python + pytest

Contents

0. The 30-second version . . . 3

1. What the backend does . . . 3

2. Stack - what is used and why . . . 4

3. File map . . . 4

4. The concept in detail . . . 5

5. API reference . . . 9

6. Configuration (.env, see .env.example) . . . 9

7. Measured, not claimed . . . 9

8. What the 38 tests actually prove . . . 10

9. Limitations - say these before a judge finds them . . . 10

10. Judge Q&A - with answers you can say out loud . . . 10

11. 60-second pitch script . . . 12

12. Glossary for the presenter . . . 13

13. Commands . . . 13

Appendix A - 20 more ideas, and exactly what each one requires . . . 13

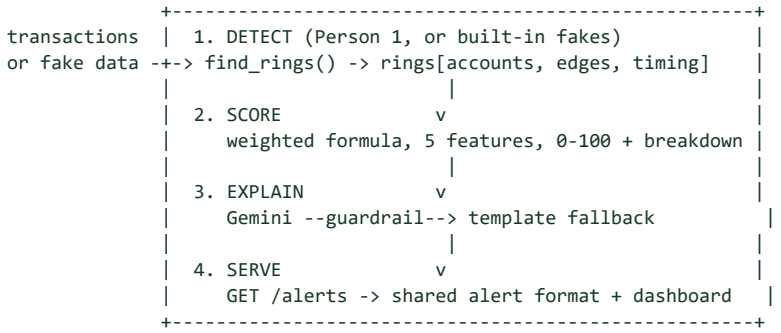
What this document is: everything the backend contains, everything it is built with, how each decision works, what has actually been measured, and the questions a judge is likely to ask - with answers you can say out loud.

Audience: the person presenting the backend (Person 2). Read section 4 twice, then section 10 the morning of the demo.

---

0. The 30-second version

> We take a network of bank transfers, find groups of accounts that look like a money-laundering ring, give each ring a **risk score out of 100 built from a formula you can inspect**, and turn it into **two plain-English sentences** - so a human analyst can act on it in seconds. The LLM only *describes* what was already computed; it never decides the score, and every number it writes is checked against the input. If the LLM is slow or down, a template takes over, so the demo never breaks.



Three design commitments drive everything:

1. **Deterministic score, probabilistic wording.** The number comes from maths we can reproduce and defend; the sentence comes from an LLM but is checked.
2. **Fail closed, never fail loud.** Missing key, timeout, 429, hallucination, broken detection -> a defined fallback, never a 500 and never a blank screen.
3. **One contract.** `ring_id, accounts, edges, total_amount, risk, explanation` - Person 3 builds the dashboard against exactly these names.

---

1. What the backend does

Stage	Input	Output	Where
Detect	transaction data (or none)	rings: accounts, edges, timing, pattern	`app/ring_source.py` -> Person 1's `detection.find_rings()` or `app/fake_rings.py`
Score	one ring	`risk` 0-100 + per-feature `score_breakdown`	`app/scoring.py`
Calibrate	score	`probability` - P(fraud), fitted not assumed	`app/calibration.py`
Decide	probability	`recommended_action` + expected loss of each action	`app/policy.py`
Explain	ring + score	two sentences + `explanation_source`	`app/explanation.py`
Serve	all rings	`AlertsResponse` JSON (+ the built dashboard)	`app/main.py`
Cache	ring facts hash / detection result	600 s / 60 s TTL in-memory	`app/cache.py`

A single request to `/alerts` walks all five. There is no database in the request path - the whole pipeline is in-process and stateless except for two TTL caches, which is what makes it fast enough to poll every 10 seconds from a dashboard.

---

2. Stack - what is used and why

Backend (runtime)

Library	Version installed	Role	Why this one
FastAPI	0.161.1	HTTP framework, routing, validation, OpenAPI	async-native, per-request validation from type hints, free Swagger at `/docs`
Uvicorn	0.54.0	ASGI server	the standard runner for FastAPI; `--reload` for development
Pydantic v2	2.14.1	the alert contract and all validation	one model = validation + serialisation + docs; Rust core is fast
httpx	0.28.1	async calls to Gemini	async + explicit per-request timeouts (the LLM must not hang the app)
python-dotenv	1.2.4	`.env` loading	keys out of the source tree; `.env` is gitignored
pytest	9.1.1	38 tests	plain asserts, no framework overhead
starlette (transitive)	1.9.0	middleware, static files, TestClient	comes with FastAPI; provides CORS and the static mount

Deliberately **not** used: no database (demo scope; SQLite audit log is the next step), no pandas/networkx in the API process (detection is Person 1's job), no ORM, no auth layer, no cache server - each would add a failure mode without adding demo value today.

Frontend (the face of the backend)

React 19 + Vite + **Cytoscape** (graph rendering) + plain CSS. Black/green theme, no UI framework. It only calls `GET /alerts`; it computes nothing that matters. Details in README.md.

External service

Google Gemini free tier (`gemini-2.5-flash`) - explanation text only. Optional: with no key, the app runs in *keyless mode* and every explanation comes from the template. The key lives server-side in `.env`, never in the browser bundle.

Python / OS

Python 3.11+ (`X | None` type syntax is used). Windows-first dev flow (`run.bat`), but nothing is Windows-specific in the code.

---

3. File map

File	Lines	What lives there
`app/main.py`	171	the FastAPI app, `/health`, `/alerts`, `/alerts/{id}`, `/policy`, CORS, static dashboard mount, request orchestration
`app/models.py`	115	`Edge`, `ScoreFeature`, `Alert`, `Decision`, `RingFacts`, `AlertsResponse` - the contract
`app/scoring.py`	165	the weighted risk formula, normalisation thresholds, all five features
`app/explanation.py`	211	Gemini call, prompt, number guardrail, reply cleaning, template layer
`app/ring_source.py`	137	detection hook, validation, failure policy, detection cache
`app/fake_rings.py`	78	two built-in rings: the planted fan-out/merge and a slower circular flow
`app/cache.py`	35	the TTL cache used for explanations and detection
`app/calibration.py`	165	Platt curve: score -> P(fraud), plus a committed-curve override
`app/labelled.py`	248	the labelled set: laundering shapes and legitimate decoys, deterministic
`app/policy.py`	121	expected-loss matrix, action choice, implied thresholds
`app/config.py`	50	every setting read from the environment in one place
`tests/`	529	38 tests: contract, scoring maths, guardrail, fallback, cache, CORS, decision layer
`frontend/`	-	React dashboard (build output served by FastAPI at `/`)

Small on purpose: ~1,500 lines of backend including docstrings. Every line can be explained under questioning.

---

4. The concept in detail

4.1 The fraud pattern being detected

A **fraud ring** is a group of accounts working together. The canonical shape in this project is **fan-out then merge**:

```
        ACC7001  --8000-->  MULE001 --7900-->
            |    --8000-->  MULE002 --7900-->
            |    --8000-->  MULE003 --7900-->   ACC7002 (sink)
            |    --8000-->  MULE004 --7900-->
            |    --8000-->  MULE005 --7900-->
            +--- --8000-->  MULE006 --7900-->        110 seconds total
```

One source splits money into near-equal chunks to **mule accounts** (throwaway receivers, often first-time), which immediately forward it to one sink. The money leaves one place, travels through six decoys, and lands in another - so no single transaction looks unusual. Per-transaction rules fail here; the *shape* is the signal. That is why the unit of analysis is a ring, not a payment.

The second built-in ring (`R2`) is a **circular flow** - A->B->C->A over 5.5 hours - which is a different laundering pattern and scores much lower (slower, fewer accounts), so the dashboard shows a real score range rather than one alarming number.

4.2 Detection input: the Person 1 contract

`ring_source.get_rings()` looks for a callable `find_rings()` in a module named by `DETECTION_MODULE` (default `detection`) at the repo root. It accepts either `find_rings()` or `find_rings(data)`, and each returned item is validated into:

```
RingFacts(
    ring_id="R1",                    # required
    accounts=["ACC7001", ...],       # required
    edges=[{ "from": ..., "to": ..., "amount": ..., "timestamp": ... }],
    total_amount=48000.0,            # optional -> summed from edges
    time_window_seconds=110.0,       # optional -> derived from timestamps
    new_accounts=7,                  # optional -> 0.5 assumed
    anomaly_score=0.87,              # optional -> adds a 6th scored feature
    pattern="fan-out then merge",    # optional -> passed to the LLM
)
```

Everything optional degrades gracefully and is *noted in the breakdown* - the API never silently invents a value without saying so in the `note` field.

Failure policy, in priority order:

1. `detection` module missing -> built-in fakes (`source: "fake"`).
2. Detection raises -> fakes, with the reason in `detection_error`.
3. Detection returns 0 rings -> `[]` honestly (or fakes if `FALLBACK_TO_FAKE_WHEN_EMPTY=true` for a live stage demo).
4. One malformed ring -> that ring is skipped and counted, the rest survive.

4.3 The risk score - the explainability core

Score = Sum (normalised feature x weight), clamped to 0-100, rounded to an int. Five features, weights from the team plan:

#	Feature	Why it matters	Weight	Normalisation (0->1)
1	**Linked accounts**	more mules = more organised	25	`accounts / 6`, capped at 1
2	**Time window**	money moved in minutes is suspicious	25	`1 - min(seconds, 600)/600`
3	**Amount similarity**	equal splits are a mule signature	20	`1 - (std-dev / mean)` of edge amounts
4	**Total amount**	higher exposure	15	`total / 50,000`, capped at 1
5	**Account newness**	throwaway first-time receivers	15	`new_accounts / accounts`

#	Feature	Why it matters	Weight	Normalisation (0->1)
6	*Anomaly score*	*Person 1's Isolation Forest, optional*	*10*	*as supplied; other five weights scale to 90 so the total stays 100*

The thresholds (6 accounts, 600 s, 50,000) are named constants at the top of `scoring.py`. Tuning happens there and nowhere else - that is a one-sentence answer to "how would you tune this?".

Worked example - the planted ring (this is the table shown in the dashboard):

Feature	Raw	Normalised	Weight	Points
Linked accounts	8	1.00	25	**25.0**
Time window	110 s	0.82	25	**20.4**
Amount similarity	0.994	0.99	20	**19.9**
Total moved	48,000	0.96	15	**14.4**
Account newness	7 of 8	0.875	15	**13.1**
Total				**92.8 -> risk 93**

The circular ring, same formula:

Feature	Raw	Normalised	Weight	Points
Linked accounts	3	0.50	25	12.5
Time window	19,800 s	0.00	25	0.0
Amount similarity	0.993	0.99	20	19.9
Total moved	35,700	0.71	15	10.7
Account newness	1 of 3	0.33	15	5.0
Total				**48.1 -> risk 48**

Read that pair out loud in the pitch: *same formula, 93 versus 48, and you can see exactly which feature caused the difference* - it is the 25 points for speed and scale that the slow 3-account ring cannot earn. That is what "explainable AI" means here, as opposed to a black box with a percentage.

Why a formula and not a model? Because a bank must be able to *reproduce and justify* a fraud score to an auditor and to a customer. A learned score can be more accurate; a weighted score can be defended line by line, and it is honest about its assumptions (each row carries a `note` when data was missing).

4.4 The explanation - three layers, one guarantee

Layer 1 - Gemini (free tier). The model receives *only* a JSON object of facts (`num_accounts`, `num_edges`, `time_window_seconds`, `total_amount`, `amount_similarity_0_to_1`, `risk_score_0_to_100`, plus optional `num_new_accounts`, `pattern`, `anomaly_score`) and a system instruction: *two sentences, no markdown, use only the numbers in FACTS, never invent/round/recompute, sentence 1 = what the accounts did, sentence 2 = what the score means and that a human decides.* `temperature 0.2`, `maxOutputTokens 200`, hard timeout **3.5 s** (connect <= 2 s).

Layer 2 - the number guardrail. Before a reply is accepted:

1. Strip markdown, quotes and whitespace; keep at most the first two sentences.
2. Reject replies under 6 words (empty or lazy output).
3. Extract **every number** from the reply with one regex (handles `48,000` and `1.8`), extract every number from the facts JSON we sent, and require each produced number to match some allowed number within `math.isclose(rel_tol=1e-3, abs_tol=0.011)`.

Any failure -> the reply is discarded and the template is used. This is the concrete answer to *"can the LLM hallucinate an amount?"* - no, because a number that is not in the input cannot leave this function, and there is a test that feeds it a hallucinated amount and asserts the template is served.

Layer 3 - the template. A fill-in-the-blank sentence built from the same payload, e.g.:

> *7 first-time receiver accounts in ring R1 were paid near-identical amounts within 1.8 minutes and forwarded the money onward, moving 48,000 in total across 12 transfers. The weighted risk score is 93 out of 100, and a human analyst makes the final call.*

It is grammatically branching (singular/plural, minutes vs seconds, "near identical" vs "varying" chunks) and uses only numbers already in the payload. `explanation_source` in the JSON says which layer produced the text, and the dashboard shows it as a `TEMPLATE` / `AI EXPLANATION` badge. **The demo is never blocked by Wi-Fi, quota or a dead API** - that is the whole point of the layer.

4.5 The shared contract

```
{
  "alerts": [
    {
      "ring_id": "R1",
      "accounts": ["ACC7001", "MULE001", "..."],
      "edges": [{ "from": "ACC7001", "to": "MULE001", "amount": 8000.0,
                  "timestamp": "2025-10-24T09:00:00Z" }],
      "total_amount": 48000.0,
      "risk": 93,
      "explanation": "7 first-time receiver accounts ...",
      "score_breakdown": [
        { "feature": "Linked accounts", "raw": 8, "normalized": 1.0,
          "weight": 25.0, "contribution": 25.0, "note": null }
      ],
      "explanation_source": "gemini" | "template",
      "generated_at": "2025-10-24T09:00:00Z"
    }
  ],
  "count": 2,
  "source": "detection" | "fake",
  "cached": true,
  "generated_at": "...",
  "detection_error": null
}
```

The six contract fields are fixed; everything else is additive and safe for the frontend to ignore. `from` is a Python keyword, so the model field is `source` but it **serialises back to** `from` and accepts `from` / `source` / `sender` on input (`AliasChoices`) - that is how Person 1 can hand over either spelling.

4.6 Caching, concurrency and why refresh is instant

Cache	Key	TTL	Saves
Explanations + alerts	SHA-256 of the ring facts	600 s	the LLM call - the expensive part
Detection result	`"rings"`	60 s	re-walking the transaction graph on every dashboard poll

- Detection is a blocking, possibly 100k-row job -> run in `asyncio.to_thread`, so the event loop keeps serving requests.
- Explanations for N rings run **in parallel** with `asyncio.gather`, not one by one, so the wall time is one LLM round-trip rather than N.
- `cached: true` in the envelope means *every* alert in that response came from the cache - a number the pitch can point at.

4.7 Failure policy, collected

Failure	What happens	What the user sees
No `GEMINI_API_KEY`	keyless mode, template only	`explanation_source: "template"`, `/health` says `"llm": "template"`
Gemini timeout / 429 / 500 / bad JSON	exception caught, template used	alert still returns, 200
Reply invents a number	guardrail rejects, template used	alert still returns, 200
Reply is empty / 3 words	length check, template used	alert still returns, 200
Detection module absent	built-in fakes	`source: "fake"`
Detection raises	fakes + reason	`detection_error` explains it
One ring malformed	skipped, others served	`detection_error` counts it
Ring has no timing/newness	defaults + explicit `note`	breakdown still sums to the score

---

4.8 The decision layer - from score to action

Two questions survive scoring: *how likely is this actually fraud?* and *what should the analyst do?* A rank answers neither.

Calibration. `app/calibration.py` fits a logistic (Platt) curve from the score to P(fraud) over `app/labelled.py`: ten laundering shapes (fan-out variants, cycles, gather-scatter, and one deliberately sloppy slow ring) and ten legitimate patterns that genuinely look ring-like (payroll, merchant settlements, rent collection, loan disbursement, refunds, savings transfers, creator payouts, gig payroll). The overlap is deliberate - a dataset with a clean gap would let isotonic regression hand out fake certainty (0 and 1); Platt smoothing keeps probabilities graded exactly where the evidence is thin.

The fitted curve, measured now: score 0 -> 0.7%, 50 -> 26%, 80 -> 79%, 93 -> 91%, 100 -> 95%. On the demo rings: **R1 -> 0.909**, **R2 -> 0.229**. That last number is the honest part - a slow circular flow with three accounts is not a 91% fraud, and now the API says so.

Expected loss. `app/policy.py` prices the three actions in units of exposure. Nothing here is hidden; `GET /policy` publishes the whole matrix:

Action	Loss if fraud	Loss if legitimate
`freeze`	0.10 (late, most already moved)	0.60 (blocked customer, complaints)
`escalate`	0.50 (delay while an analyst works)	0.05 (analyst time)
`watch`	1.00 (everything lost)	0.00

`EL(action) = p x fraud_loss + (1 - p) x legit_loss`, and the recommended action is the cheapest. Thresholds are **not constants in code** - they fall out of the matrix: 0.091 for review, 0.579 for freezing, recomputed by `GET /policy` whenever the matrix changes.

Ring	P(fraud)	freeze	escalate	watch	Recommended
R1 fan-out	0.909	**0.15**	0.46	0.91	**freeze**
R2 circular	0.229	0.49	**0.15**	0.23	**escalate**

Every alert now carries `probability` and `decision` (`recommended_action`, `expected_loss`, `policy_version`, `rationale`), the rationale is generated from the same numbers, and both the policy and the calibration versions are stamped in `/health` **and included in the cache key** - so an alert scored under old maths is never served after the maths changes.

Honest scope: losses are normalised to one unit of exposure, so a small ring and a very large ring with the same probability get the same action. Scaling the matrix by exposure is the next step, not something this version claims.

5. API reference

Method	Path	Returns
GET	/alerts	envelope with all alerts, source, cached, detection_error
GET	/alerts/{ring_id}	one alert; 404 for an unknown id
GET	/policy	cost matrix, implied thresholds, calibration curve and both versions
GET	/health	status, active LLM layer, cache TTL, detection target
GET	/docs	Swagger UI (try every endpoint live)
GET	/openapi.json	machine-readable schema
GET	/	the built React dashboard, when frontend/dist exists

/health is deliberately cheap: it never runs detection, so it is safe for a monitor or a load balancer.

---

6. Configuration (.env, see .env.example)

Variable	Default	Meaning
GEMINI_API_KEY	*(empty)*	empty = keyless template mode
GEMINI_MODEL	gemini-2.5-flash	model id
LLM_TIMEOUT_SECONDS	3.5	hard budget per explanation
LLM_ENABLED	true	force template mode without removing the key
CACHE_TTL_SECONDS	600	explanation cache life
DETECTION_CACHE_TTL_SECONDS	60	detection cache life
DETECTION_MODULE / DETECTION_FUNCTION	detection / find_rings	Person 1 hook
FALLBACK_TO_FAKE_WHEN_EMPTY	false	protect a live demo from an empty detection run
FAKE_ALERTS_PATH	data/fake_alerts.json	Person 4's fake data (falls back to built-ins)
CORS_ORIGINS	*	allowed origins

.env is gitignored; only .env.example is committed. Settings are read as config.X (module attribute) so tests can monkeypatch them.

---

7. Measured, not claimed

Everything below was measured on this machine, template mode, warm process:

Measurement	Result	How
/alerts first call (cold, template layer)	**~9 ms**	curl -w '%{time_total}' against a fresh process
/alerts cached	**~4 ms**	same, subsequent calls, cached: true
Test suite	**38 passed in ~0.2 s**	pytest -q
Production frontend build	**~0.4 s**, exit 0	npm run build
Explanation cost when Gemini is enabled	bounded by **3.5 s**, typically 1-2 s	LLM_TIMEOUT_SECONDS hard cap; cached afterwards at ~4 ms

Say the honest version on stage: *"the formula is microseconds; the LLM is the only slow part, it is capped at 3.5 seconds, and it runs once per ring because of the cache."* Do not claim a Gemini latency you have not measured with your own key - measure it once when the key is in, and quote that.

---

8. What the 38 tests actually prove

Area	Tests prove
Contract	`/alerts` returns exactly the shared format; `/docs` is served; CORS allows the dashboard origin
Risk score	base weights sum to 100; the planted ring scores 90+; the score **equals** the weighted sum of its own breakdown; a payroll decoy scores far below the planted ring; the optional anomaly feature keeps the total at 100; missing timing/newness are assumed and noted; single-edge and empty rings do not crash
Guardrail	a hallucinated number is rejected; comma formatting and rounding are accepted; a clean Gemini reply is used; a hallucinated one falls back to template; a timeout falls back; replies are truncated to two sentences; markdown is stripped
Fallbacks	keyless mode uses the template
Caching	the second request is `cached: true`; both caches populate
Decision	the curve is monotone and stays inside 0-1; the chosen action is the cheapest at every probability; thresholds are ordered; `/policy` publishes the matrix; the cache key changes when the policy does
Endpoints	`/health` reports the active layer; `/alerts/{id}` works and 404s

That is the "we can prove it" section: run `pytest -q` live if a judge asks.

---

9. Limitations - say these before a judge finds them

1. **The data is synthetic.** Two built-in rings plus optional `data/fake_alerts.json`. The *method* transfers to real transaction data; the numbers do not.
2. **No persistence.** Explanations and detection live in memory; the dashboard's Freeze/Escalate buttons are session-only. The audit trail (`POST /action` -> SQLite row of who/what/when, then an Action history list) is designed but not built - it is the next 30-minute piece, and it is what a bank would ask for first.
3. **No auth, no rate limiting.** Fine for a demo behind a tunnel; a real deployment needs an API key/OAuth, per-client quotas and PII redaction.
4. **Account IDs go to the LLM.** In the demo they are fake. In production the payload would be pseudonymised (hashed ids, bucketed amounts) - worth saying unprompted, it shows you thought about the DPDP Act.
5. **Amount similarity is pattern-level, not discriminative.** A circular ring of near-equal transfers also scores high on that feature (both rings earn ~20/20). The separation comes from accounts + window. A learned or calibrated model, or normalising similarity against the *account's* history, is the fix.
6. **Thresholds are hand-set** (6 accounts, 600 s, 50k) rather than fitted. They are constants in one file precisely so they can be recalibrated on labelled data later.
7. **One process, in-memory cache.** Multiple uvicorn workers would each have their own cache - Redis is the upgrade path. Detection at 100k rows in a thread is fine for a demo, not for a bank's volume.
8. **The calibration is fitted on synthetic labels.** The curve is honest about where the evidence is thin, but it would be refitted on a bank's confirmed cases before anyone acted on the percentages.
9. **Expected losses ignore exposure size.** The matrix is normalised, so two rings with the same probability get the same action regardless of amount; scaling by exposure (or by a bank's risk appetite) is the next iteration.
8. **The LLM is not the scorer.** If a judge wants "AI", the answer is: AI writes the narrative and can contribute one feature; the decision path stays auditable.

---

10. Judge Q&A - with answers you can say out loud

Concept & novelty

Q: What exactly is the problem you are solving? Money laundering is not one big transaction, it is a *pattern* across accounts: split money into equal chunks, push it through throwaway accounts, merge it elsewhere. Per-transaction rules miss it because every single payment looks normal. We detect the group, score it, and explain it in language a human can act on.

Q: What is new here? Everyone has a fraud dashboard. Three things, in order: (1) the score is not a black box - the API returns the per-feature maths behind it; (2) the LLM is *constrained* - a code guardrail rejects any number not in the input, which is the part most teams skip; (3) the system is designed to survive failure - LLM down, key missing, detection broken, and the dashboard still shows something truthful.

Q: Where does the AI actually decide something? It does not decide. It summarises. That is deliberate: a fraud score must be reproducible for audit and for the customer. The AI's job is the 20% that consumes analyst time - turning a score table into a sentence.

Q: Is that not just a prompt? The prompt is the smallest part. The engineering is the contract, the scoring maths, the concurrency, the caching, and the guardrail that makes the prompt safe to ship.

The score

Q: Why 93? Show me. Eight accounts (full 25), 110 seconds (20.4 of 25), near-identical splits (19.9 of 20), 48,000 moved (14.4 of 15), 7 of 8 first-time receivers (13.1 of 15) - 92.8, rounded to 93. It is in the response as `score_breakdown`, and it is on screen.

Q: You say 93, the screen says 91% - which is it? Both, and they answer different questions. 93 is the weighted score: a ranking signal whose breakdown an auditor can check. 91% is the calibrated probability that a ring with that score is genuinely fraudulent, fitted on labelled data - that is the number the cost model consumes. The API returns both.

Q: Where is the threshold - 60? 80? There isn't one. The cost matrix implies the break-evens: 0.09 for sending it to an analyst and 0.58 for freezing. Change a cost and `GET /policy` shows the new thresholds; nothing is buried in an `if` statement.

Q: Who chose those weights? We did, from domain reasoning about mule behaviour - many accounts, fast, equal splits, high exposure, new receivers. The important engineering point is that they are five named constants in one file, so tuning is a one-line change, and a future version can fit them on labelled data instead.

Q: Couldn't you just train a model? Detection already can be ML (Person 1's graph/anomaly work; an optional Isolation Forest score plugs in as a sixth feature worth 10 points, and the other weights scale to 90 so the total still sums to 100). We kept the *aggregation* transparent because that is the layer a bank has to justify.

Q: What about false positives - a payroll account paying 50 people? There is a test for exactly that: a payroll-style decoy scores far below the planted ring, because payroll has many accounts but no tight window, no merge, and established accounts. And nothing is auto-blocked - a human presses Freeze or Escalate.

Q: What if a real ring scores 40? Thresholds are tunable per deployment; the breakdown tells the analyst *why* the score is low (usually: not enough accounts or too slow a window), which is often the correct answer for a legitimate business account.

The LLM

Q: Can it hallucinate? It cannot put an invented number in front of a user: every number in the reply is regex-extracted and must match a number from the facts we sent (tolerance 0.011 for rounding). Fail -> the template is served instead. There is a test that does this deliberately.

Q: What happens if the LLM is down during the demo? A 3.5-second timeout, an exception handler, and the template layer - the same alert, the same score, slightly more mechanical wording. `/health` shows which layer is active, and the dashboard badges it `TEMPLATE`. The demo cannot break.

Q: Why Gemini and not GPT/Claude/local? Free tier for a hackathon, one HTTP call, no SDK weight. The code is one function (`_call_gemini`) with a provider-shaped payload; swapping providers is an afternoon, and the guardrail/template layers do not change at all. A local Ollama model is the offline backup if the venue Wi-Fi dies.

Q: Cost per alert? Zero on the free tier; otherwise one short call per ring per 10 minutes, cached. At scale: batch explanations, cheaper model, or pre-generate on write.

Q: Is sending account data to an LLM allowed? For the demo the ids are synthetic. In a real deployment we would send pseudonymised ids and bucketed amounts, or run a local model - and that is a compliance decision, not just an engineering one.

Architecture & scale

Q: What happens with 100k transactions? Detection runs in a worker thread so the event loop keeps serving; its result is cached for 60 s; explanations run in parallel and are cached for 10 minutes. Measured now: ~9 ms cold, ~4 ms cached at the API. At real volume the change is architectural: batch/streaming detection, a graph store (Neo4j/Neptune), Redis for shared cache, a queue for explanations.

Q: Why FastAPI? Async I/O for the LLM call, validation and OpenAPI from type hints, and a `/docs` page we could hand to a judge. Flask would need all of that added by hand.

Q: Why no database? Nothing in the request path needs one, and a demo that cannot fail on a DB connection is a better demo. The one thing that genuinely wants persistence is the analyst audit trail - SQLite, next step.

Q: How would you deploy this? Container behind a reverse proxy, uvicorn workers, Redis for the cache, secrets in a vault, and the LLM behind a circuit breaker. We already demo it through a Cloudflare tunnel on one URL.

Q: How do you test it?38 pytest tests, no mocks of our own logic - only the LLM HTTP call is stubbed. They assert the maths, the contract, the guardrail, the fallbacks, the calibration curve and the policy. Run them live.

Product & safety

Q: Who uses this and what do they do with it? A fraud analyst. They see a ranked queue, click a ring, read two sentences and a breakdown, then Freeze (contain the accounts) or Escalate (hand to an investigator for a case/STR). The machine narrows; the human decides.

Q: What is the regulatory hook? Suspicious Transaction Reports - banks must file with FIU-IND in India. The same facts that produce the explanation are what an analyst would type into a report, so the natural extension is an auto-drafted STR clearly labelled *draft*.

Q: What would you build next with more time?

1. Persistent audit trail (`POST /action` + SQLite + Action history), 2. drafted STR, 3. SSE streaming so rings appear live as transactions replay, 4. calibration on labelled data, 5. pseudonymised LLM payloads.

   Q: What did *you* build? Person 2's scope: the shared contract, the transparent scoring with breakdown, the three-layer explanation with the number guardrail, the caching/concurrency, the API and its failure policy, and the 26 tests. Person 1 owns detection, Person 3 the dashboard, Person 4 the data and pitch.

---

11. 60-second pitch script

> "Banks look at one transaction at a time. Laundering doesn't work that way - it splits money into equal chunks, pushes it through throwaway accounts, and merges it somewhere else. Every individual payment looks boring. So we detect the *ring*, then score it out of a hundred with a formula you can read: accounts involved, how fast the money moved, how equal the splits were, how much was moved, how new the receivers are. This ring gets 93 - and here is exactly where each point came from. No black box. Then an LLM turns that into two sentences an analyst can act on. But we do not trust it with numbers: every figure it writes is checked against the input, and if it invents one - or if the API is slow, or down - a template takes over. The demo cannot break, and the model cannot invent an amount. Human in the loop: the system explains, the analyst decides. Freeze the accounts, or escalate for investigation."

---

12. Glossary for the presenter

Term	Meaning
Mule account	a throwaway account used to receive and forward laundered money
Fan-out / merge	one source splits to many accounts, which consolidate into one sink
Circular flow	money moves A->B->C->A to obscure its origin
Isolation Forest	an anomaly-detection model that isolates outliers; optional 6th feature here
Coefficient of variation	standard deviation / mean; low value = equal-sized transfers
TTL cache	in-memory store where entries expire after N seconds
Guardrail	code that validates LLM output against ground truth before it is used
Keyless mode	running with no LLM key, template explanations only
STR / FIU-IND	Suspicious Transaction Report / India's Financial Intelligence Unit
Human-in-the-loop	the automated system advises; a person authorises action
Platt scaling	fitting a logistic curve to map a score onto a probability
Expected loss	p(fraud) x cost if fraud + (1-p) x cost if legitimate
Break-even threshold	the probability where two actions cost the same

---

13. Commands

```
:: everything, one file
run.bat                 :: setup + build + serve at http://127.0.0.1:8000
run.bat tunnel          :: same, plus a public trycloudflare.com link

:: manual
.\.venv\Scripts\activate.bat
pip install -r requirements.txt
pytest -q                               :: 38 passed
uvicorn app.main:app --reload --port 8000
curl http://127.0.0.1:8000/health       :: which explanation layer is live
curl http://127.0.0.1:8000/alerts       :: the JSON the dashboard consumes
```

See RUN.md for setup troubleshooting and README.md for the team-facing contracts.

Backend: FastAPI + Pydantic + httpx + Gemini (optional) + pytest. ~1,500 lines, 38 tests, two caches, three explanation layers, one calibration curve, one published cost matrix, zero black boxes.

Appendix A - 20 more ideas, and exactly what each one requires

Everything below is **additive**: no idea here requires changing the six contract fields, and each one is scoped so it can ship in one sitting. Sorted roughly by impact / effort. "Requires" means the concrete artefacts you must create - not vibes.

A. Score integrity (highest value, smallest change)

1. Counterfactual: "what would make this ring *not* flag?" The inverse of the breakdown panel. For each feature, compute the value that would drop the score under the alert threshold, and return the cheapest one ("if the window had been 14 minutes instead of 110 seconds, this ring scores 58").

> *Requires:* a pure function in `scoring.py` that re-runs the formula with one feature overridden (30 lines), `GET /alerts/{id}/counterfactual`, 2 tests.
>
> *~40 min. Said on stage:* "we don't just say why it fired, we say what the difference was."

2. Weight sensitivity report. Perturb every weight +/-20%, re-score both rings, and show the score barely moves. That is evidence the score is not brittle - a question judges ask about hand-set weights.

> *Requires:* a script `bench/sensitivity.py` printing a table, plus `bench/sensitivity.md` committed with the output. No API change, no deps.
>
> *~30 min, no risk to the demo.*

3. Scoring version + input fingerprint on every alert. Each alert carries `scoring_version` (hash of the weights and thresholds) and `input_hash` (hash of the facts). Any alert can be reproduced and any score can be tied to the exact formula that produced it - the audit answer to "can you reproduce last week's decision?".

> *Requires:* two computed fields in `models.py`, one hash helper, a test asserting the version changes when a weight changes. *~45 min.*

4. Confidence band / abstention. When the features disagree - high amount similarity but a 6-hour window - mark `confidence: "low"` and `needs_review: true` instead of pretending the score is precise. Honest systems admit uncertainty; almost no hackathon project does.

> *Requires:* a small dispersion rule in `scoring.py` (e.g. std-dev of normalised features), one new optional field on `Alert`, 2 tests. *~1 h.*

B. Evidence from the graph, not just numbers

5. Evidence subgraph. Return the *minimum* set of edges that justifies the ring: the account with the highest fan-in (the merge point), the equal-amount group, and the fastest chain. The dashboard highlights only those edges, which turns a busy graph into a finding.

> *Requires:* a function over `RingFacts.edges` (in-degree max + BFS shortest path), `evidence_edges: list[int]` (indices) on the alert, frontend highlight.
>
> *~1.5 h; the single most visual addition.*

6. Account aliasing (entity resolution). Two accounts that share a phone number, device id or IFSC are usually one actor. Union-find over an alias table doubles the effective ring size and catches rings splitting across more accounts.

> *Requires:* `data/account_aliases.csv`, union-find (~40 lines), applied before scoring, a test with two aliased accounts. *~2 h, needs data from Person 4.*

7. Smart-decoys test set. Rings built to *look* fraudulent but be legitimate, or vice versa: 3 accounts with a 9-minute window and round numbers (suspicious-looking, low score); a payroll with 50 receivers (high accounts, low everything else); rent splits between family members. Assert the legitimate ones stay under the threshold.

> *Requires:* 5-6 extra `RingFacts` fixtures in `tests/`, assertions per case.
>
> *~1 h, pure data - no new dependencies.*

C. Analyst workflow (what a bank actually buys)

8. Audit trail with a tamper-evident chain. `POST /action {ring_id, action, actor, note}` -> SQLite row `(id, ts, ring_id, action, actor, note, prev_hash, row_hash)`. Each row hashes the previous row, so deleting or editing history breaks the chain and `GET /actions/verify` says so. That is the difference between "a log" and "evidence".

> *Requires:* `sqlite3` (stdlib), `app/audit.py`, two endpoints, 3 tests, and the dashboard's Freeze/Escalate finally persisting. *~1.5 h.*

9. Auto-drafted STR (suspicious transaction report). `POST /alerts/{id}/report` -> `{subject, grounds[], narrative, recommended_action, status: "draft"}` built from the same facts, through the same number guardrail, so the draft cannot contain a fabricated figure. Clearly labelled draft, one click to download as `.md`.

> *Requires:* a second prompt (structured JSON output), a Pydantic report model, guardrail reuse, template fallback, 2 tests. *~1.5-2 h; strongest "real workflow" story (FIU-IND).*

10. Analyst feedback -> calibration. `POST /feedback {ring_id, verdict: true_positive|false_positive}` stored in SQLite; `GET /stats` reports counts and a running precision per score band (80-100, 60-80, ...). No retraining needed for the demo - the band table alone is a credible answer to "how do you know it works?".

Requires: SQLite (shared with section 8), 2 endpoints, banding logic, 2 tests.

~1.5 h; optionally tune one weight from the tally and say so.

11. Case notes on a ring. `PATCH /alerts/{id}/notes` with an analyst comment, shown in the panel and included in the report draft. Small, but it makes the dashboard feel like a tool an analyst returns to rather than a visualisation.

Requires: SQLite table, one endpoint, textarea in the panel. *~45 min.*

D. Live demo and operations

12. Server-Sent Events replay. `GET /alerts/stream` pushes each alert as its ring completes while a transaction replay runs, so the graph grows on screen instead of appearing pre-baked.

Requires: `sse-starlette`, an async generator over Person 1's replay output, frontend `EventSource`, a test that asserts the first event shape. *~2 h, and it pairs with Person 1's replay and Person 3's animation.*

13. Per-stage timings in the response. `timings: {detection_ms, score_ms, llm_ms, cache}` on the envelope. The latency slide then writes itself, and a judge can see exactly where the 1.5 s went.

Requires: `time.perf_counter()` around three calls in `main.py`, one model.

~30 min - cheapest credible performance claim you can add.

14. Request id + structured logs. Middleware assigns `X-Request-Id` and logs one JSON line per request with status and duration; the id is returned in the response. Makes the tunnel/LAN demo debuggable in front of judges instead of guessing.

Requires: one middleware, a JSON log formatter, header echo. *~40 min.*

15. Single-flight for explanations. Five dashboard tabs refresh at once -> five identical LLM calls today. Keep an `asyncio` task registry keyed by the facts hash so concurrent requests await the same in-flight call.

Requires: a dict of tasks in `main.py`, one test with `asyncio.gather` firing two requests and asserting a single LLM call. *~45 min; a real systems answer.*

16. Rate limit + optional API key for the public tunnel. Token bucket per IP (e.g. 60 req/min), `X-API-Key` check when `API_KEY` is set, `429`/`401` with `Retry-After`. Needed the moment the `trycloudflare.com` link is shared beyond the team.

Requires: one middleware, two env vars, 2 tests. *~45 min.*

E. Privacy, reliability, polish

17. Pseudonymised LLM payload. Hash account ids with an HMAC salt and bucket amounts before they leave the process, keeping the mapping local so the explanation can still name accounts. Directly answers the DPDP-Act question.

Requires: `app/pseudonym.py` (stdlib `hmac`/`hashlib`), salt in `.env`, a test asserting no raw account id appears in the request body sent to the (mocked) LLM. *~45 min.*

18. Fail-fast config validation. On startup, check the timeout is sane, the CORS list parses, the fake-alert file is readable, and print one clear line per problem instead of failing on the first request.

Requires: a `lifespan` handler with checks, 3 tests. *~40 min.*

19. Multi-language explanations. `?lang=hi|en` on `/alerts` - the prompt is rewritten per language, the guardrail is unchanged because the numbers are still the facts. Localisation is a real requirement for Indian banking deployments.

> *Requires:* a language map, one extra prompt variant, template strings per language, a test that the guardrail still passes Hindi output. *~1 h.*

20. Reproducibility + CI hygiene. `Dockerfile` (python:3.12-slim, non-root, uvicorn), `docker compose` with the built frontend, and a GitHub Actions workflow running `pytest -q` plus the frontend build on push. Judges rarely ask, but "it runs from a clean checkout" is what makes the repo reusable after the event.

> *Requires:* `Dockerfile`, `compose.yaml`, `.github/workflows/ci.yml`, a README line. *~1 h; zero runtime risk.*

If you only have 90 minutes left

Order	Idea	Why this one
1	**#8 audit trail with hash chain**	converts two dead buttons into a persisted, verifiable workflow - the question every bank judge asks
2	**#13 per-stage timings**	turns "it's fast" into three measured numbers on the slide
3	**#1 counterfactual**	the most distinctive explainability line in the whole pitch
4	**#5 evidence subgraph**	visible on screen in five seconds, minimal code
5	**#3 scoring version + fingerprint**	45 minutes for the strongest audit answer you can give

Ideas to resist

- **A learned scorer.** It breaks the reproducibility story unless you keep the breakdown, and you cannot defend a weight you cannot see.
- **A real database in the request path.** Adds a failure mode to a demo that currently cannot fail on infrastructure.
- **Fine-tuning an LLM.** Cost, latency and evaluation burden, for a task where two sentences from a good prompt plus a guardrail already pass.
- **Auto-blocking accounts.** The pitch is human-in-the-loop; automation removes your best answer to "what if the model is wrong?".
- **Kafka/streaming infrastructure.** Say it as the production roadmap, do not build it at a hackathon.

All twenty ideas use only what is already installed (`fastapi`, `pydantic`, `httpx`, `pytest`) plus Python's standard library - `sqlite3`, `hmac`, `asyncio` - with two optional exceptions (`sse-starlette` for #12, Docker for #20). Nothing here requires a paid service or a new account.